VIRICFICATION

ASSIGNMENT 3
(UART TRANSMITTER)

Under Supervision of:

Eng. Hassan Khaled

Submitted by:

Ahmed Belal

Table of Contents

Overview 2
Design 3
Verification 4
uart_interface Module: 4
enum_pkg Module: 5
uart_packet Module: 6
uart_tb Module: 7
uart_top Module: 12
Transcript 13
Wave Form 16
Coverage 18
Do File 18

Overview

This assignment focuses on the design and verification of a UART transmitter using SystemVerilog. The design implements the UART transmission process including the start bit, data bits, optional parity bit, and stop bit using a finite state machine. A verification environment was developed using a SystemVerilog testbench that generates constrained-random stimulus, drives inputs to the DUT through an interface, and compares the transmitted output with a golden model to ensure correctness. Functional coverage and code coverage were also implemented to evaluate how thoroughly the design was tested under different data values and parity configurations.

Design

Two issues were found in the UART transmitter design. The first was in the parity logic, where the even and odd parity calculations were reversed. This was corrected so that even parity uses the inverted XOR of the data bits and odd parity uses the direct XOR.

The second issue was the transmission order of the data bits. The design originally sent the MSB first, but UART requires LSB-first transmission, so the logic was fixed to transmit the least significant bit first.

```
module uart_tx (uart_interface UART_I);

    // State machine states
    localparam IDLE   = 3'd0,
               START  = 3'd1,
               DATA   = 3'd2,
               PARITY = 3'd3,
               STOP   = 3'd4;

    reg [2:0] state;
    reg [3:0] bit_cnt;
    reg [7:0] shift_reg;
    reg       parity_bit;

    always @(posedge UART_I.clk or negedge UART_I.rst_n) begin
        if (!UART_I.rst_n) begin
            state     <= IDLE;
            UART_I.tx        <= 1'b1;  // idle is high
            UART_I.tx_busy   <= 1'b0;
            shift_reg <= 8'd0;
            bit_cnt   <= 4'd0;
            parity_bit <= 1'b0;
        end else begin
            case (state)
                IDLE: begin
                    UART_I.tx <= 1'b1;
                    UART_I.tx_busy <= 1'b0;
                    if (UART_I.tx_start) begin
                        shift_reg <= UART_I.data_in;
                        bit_cnt <= 4'd0;
                        parity_bit <= (UART_I.even_parity) ? ~(^UART_I.data_in) : (^UART_I.data_in);
                        UART_I.tx_busy <= 1'b1;
                        state <= START;
                    end
                end

                START: begin
                    UART_I.tx <= 1'b0; // start bit
                    state <= DATA;
                end

                DATA: begin
                    UART_I.tx <= shift_reg[0];
                    shift_reg <= shift_reg >> 1;
                    bit_cnt <= bit_cnt + 1;
                    if (bit_cnt == 4'd7) begin
                        if (UART_I.parity_en)
                            state <= PARITY;
                        else
                            state <= STOP;
                    end
                end

                PARITY: begin
                    UART_I.tx <= parity_bit;
                    state <= STOP;
                end

                STOP: begin
                    UART_I.tx <= 1'b1; // stop bit (always 1)
                    state <= IDLE;
                end
            endcase
        end
    end

endmodule
```

Verification

uart_interface Module:

The uart_interface defines the communication signals between the UART transmitter and the testbench. It groups all input and output signals such as clock, reset, data input, parity control signals, transmit output, and busy flag into a single interface, simplifying the connection between the design and the testbench.

```
interface uart_interface(input bit clk);

logic        rst_n ;
logic        tx_start ;
logic [7:0]  data_in ;
logic        parity_en ;     // 1 = enable parity
logic        even_parity ;   // 1 = even, 0 = odd
logic        tx ;
logic        tx_busy ;

endinterface
```

enum_pkg Module:

The enum_pkg package defines an enumerated data type used to represent the UART parity modes. It includes three possible values: ODD, EVEN, and NO_PARITY. This package is imported in other modules and classes to standardize how parity modes are represented and used during stimulus generation and verification.

```
package enum_pkg ;

    typedef enum {ODD , EVEN , NO_PARITY} uart_e;

endpackage
```

uart_packet Module:

The uart_packet class is used to generate randomized stimulus for the UART transmitter. It contains randomized variables for the input data and the parity mode, along with constraints to control which parity type is used. The class also includes a covergroup to collect coverage for the data values, parity modes, and their combinations, ensuring that different stimulus scenarios are exercised during verification.

```
import enum_pkg::* ;

class uart_packet;
    randc logic  [7:0]   data_in ;
    rand uart_e         enum_r ;

    constraint Odd_parity { enum_r == ODD ; }
    constraint Even_parity { enum_r == EVEN ; }
    constraint No_parity { enum_r == NO_PARITY ; }


    covergroup U_cg ;
        Data:   coverpoint data_in ;
        Parity: coverpoint enum_r  ;
        Crossing: cross data_in , enum_r ;

    endgroup

    function new();
        U_cg = new() ;
    endfunction

    task print ();
        $display("Stimulus Values in Packed are:") ;
        $display("Data = %b , Parity = %s" , data_in , enum_r.name) ;
    endtask

endclass
```

uart_tb Module:

The uart_tb module implements the verification environment for the UART transmitter. It generates constrained-random stimulus using the uart_packet class to produce different data values and parity modes. The testbench builds a golden model of the expected UART frame (start bit, data bits, optional parity bit, and stop bit), drives the inputs to the DUT through the interface, and captures the transmitted serial output. The received bits are stored in a queue and compared with the golden model using assertions to verify the correctness of the design under different parity configurations.

```
import enum_pkg::* ;
`include "uart_packet.sv"

module uart_tb(uart_interface UART_I);

//-------------------- Defining Signals --------------------//
bit         golden_model [$] ;
bit         actual_queue [$] ;
int         counter ;       // Is calculated during generating stimulus and used in storing the Actual Data from DUT

logic [7:0] data_in_r ;     // The randomized data from the class
logic       parity_en_r ;
logic       even_parity_r ;
int         mode ;


uart_packet U ;             // Defining the class


```

```
//-------------------- Generate Stimulus Function --------------------//
task automatic generate_stimulus(ref bit golden_model [$] , ref bit actual_queue [$] , int mode);
    @(negedge UART_I.clk) ;

    golden_model.delete() ;
    actual_queue.delete() ;
    U = new() ;

    // This case difine the parity condition we are using randomizing on
    case (mode)
        ODD: begin
            U.Odd_parity.constraint_mode(1) ;
            U.Even_parity.constraint_mode(0) ;
            U.No_parity.constraint_mode(0) ;
            $display("Testing ODD Parity Mode");
        end

        EVEN: begin
            U.Odd_parity.constraint_mode(0) ;
            U.Even_parity.constraint_mode(1) ;
            U.No_parity.constraint_mode(0) ;
            $display("Testing EVEN Parity Mode");
        end

        NO_PARITY: begin
            U.Odd_parity.constraint_mode(0) ;
            U.Even_parity.constraint_mode(0) ;
            U.No_parity.constraint_mode(1) ;
            $display("Testing NO Parity Mode");
        end
    endcase

    assert (U.randomize()) else $stop ;
    U.U_cg.sample() ;

    data_in_r = U.data_in ;

    // This case difine the parity condition we are using
    case (U.enum_r)
        NO_PARITY: begin
           parity_en_r  = 0;
           even_parity_r = 0; // don't care
        end

        EVEN: begin
           parity_en_r  = 1;
           even_parity_r = 1;
        end

        ODD: begin
           parity_en_r  = 1;
           even_parity_r = 0;
        end
    endcase
endtask
```

```
//-------------------- Golden Model Task --------------------//
task automatic golden_model_task (ref bit golden_model [$] , ref int counter );
    @(negedge UART_I.clk) ;

    counter = 0 ;
    golden_model [counter] = 0 ;

    for ( counter = 1 ; counter < 9 ; counter++ ) begin
        golden_model[counter] = data_in_r[counter - 1] ;
    end

    if (parity_en_r) begin
        if (even_parity_r) begin
            golden_model [counter] = ~(^data_in_r) ;
            counter = counter + 1 ;
        end
        else begin
            golden_model [counter] = (^data_in_r) ;
            counter = counter + 1 ;
        end
    end

    golden_model [counter] = 1 ;

endtask


//-------------------- Drive Stimulus Task --------------------//
task automatic drive_stim ();
    @(negedge UART_I.clk) ;

    if(!UART_I.tx_busy) begin
        UART_I.data_in = data_in_r ;
        UART_I.parity_en = parity_en_r ;
        UART_I.even_parity = even_parity_r ;

        UART_I.tx_start = 1 ;
        @(negedge UART_I.clk) ;
        UART_I.tx_start = 0 ;
    end
    else begin
        $error("The Uart is Busy") ;
        return ;
    end

endtask


```

```
//-------------------- Collect Output Data Task --------------------//
task automatic collect_output_data (ref bit actual_queue [$] );
    wait (UART_I.tx == 0);

    for ( int i = 0 ; i < counter + 1 ; i++ ) begin
        @(negedge UART_I.clk) ;
        actual_queue[i] = UART_I.tx ;
    end
endtask


//-------------------- Check Results Task --------------------//
task automatic results_check( const ref bit golden_model [$] , const ref bit actual_queue [$] );
    @(negedge UART_I.clk) ;

    $display("Golden queue = %p", golden_model);
    $display("Actual queue = %p", actual_queue);

    foreach(golden_model[i]) begin
        assert (golden_model[i] == actual_queue[i])
        else begin
            $error("DUT Data = %b , Expected Data = %b" , actual_queue[i] ,  golden_model [i] );
            $stop ;
        end
    end

    // foreach(golden_model[i]) begin
    //     if ( golden_model [i] == actual_queue[i] ) begin

    //         $display("DUT Data = %b , Expected Data = %b" , actual_queue[i] ,  golden_model[i]) ;

    //     end else begin

    //         $error("DUT Data = %b , Expected Data = %b" , actual_queue[i] ,  golden_model [i] ) ;
    //         $stop ;

    //     end
    // end
endtask
```

```
initial begin

UART_I.rst_n = 0 ;
@(negedge UART_I.clk) ;
UART_I.rst_n = 1 ;
@(negedge UART_I.clk) ;

counter = 0 ;

//-------------------- Test all data_in values on ODD PARITY --------------------//
repeat(350) begin
mode = ODD ;
generate_stimulus (golden_model , actual_queue , mode);
golden_model_task (golden_model , counter);
drive_stim ();
collect_output_data (actual_queue);
results_check (golden_model,actual_queue);
$display("==========================================================================================");
end
$display("");
$display("Test for ODD Data passed");
$display("");
$display("==========================================================================================");


//-------------------- Test all data_in values on EVEN PARITY --------------------//
repeat(400) begin
mode = EVEN ;
generate_stimulus (golden_model , actual_queue , mode);
golden_model_task (golden_model , counter);
drive_stim ();
collect_output_data (actual_queue);
results_check (golden_model,actual_queue);
$display("==========================================================================================");
end

$display("");
$display("Test for EVEN Data passed");
$display("");
$display("==========================================================================================");

//-------------------- Test all data_in values on NO PARITY --------------------//
repeat(330) begin
mode = NO_PARITY ;
generate_stimulus (golden_model , actual_queue , mode);
golden_model_task (golden_model , counter);
drive_stim ();
collect_output_data (actual_queue);
results_check (golden_model,actual_queue);
$display("==========================================================================================");
end

$display("");
$display("Test for NO PARITY Data passed");
$display("");
$display("==========================================================================================");
$display("");
$display("The design is functioning correctly") ;
$display("");
$display("==========================================================================================");
$display("");


$stop ;
end
endmodule
```

uart_top Module:

The UART_TOP module serves as the top-level module that connects the testbench and the UART transmitter design. It generates the system clock and instantiates the UART interface, the DUT (uart_tx), and the testbench (uart_tb). The interface is used to connect the signals between the DUT and the testbench.

```
module UART_TOP();

bit clk ;

//-------------------- Clock Generation --------------------//
initial begin
    clk = 0;
    forever #5ns clk = ~clk;
end

uart_interface UART_I (clk) ;
uart_tx DUT (UART_I) ;
uart_tb TB (UART_I) ;

endmodule
```

Transcript

- **Snippet from ODD PARITY Test**

```
# Testing ODD Parity Mode
# Golden queue = '{0, 1, 0, 1, 1, 1, 1, 0, 0, 1, 1}
# Actual queue = '{0, 1, 0, 1, 1, 1, 1, 0, 0, 1, 1}
# ==========================================================================================
# Testing ODD Parity Mode
# Golden queue = '{0, 1, 0, 0, 1, 0, 1, 0, 0, 1, 1}
# Actual queue = '{0, 1, 0, 0, 1, 0, 1, 0, 0, 1, 1}
# ==========================================================================================
# Testing ODD Parity Mode
# Golden queue = '{0, 1, 1, 0, 1, 1, 0, 0, 0, 0, 1}
# Actual queue = '{0, 1, 1, 0, 1, 1, 0, 0, 0, 0, 1}
# ==========================================================================================
# Testing ODD Parity Mode
# Golden queue = '{0, 0, 1, 1, 0, 1, 0, 1, 1, 1, 1}
# Actual queue = '{0, 0, 1, 1, 0, 1, 0, 1, 1, 1, 1}
# ==========================================================================================
# Testing ODD Parity Mode
# Golden queue = '{0, 1, 1, 0, 1, 0, 1, 1, 1, 0, 1}
# Actual queue = '{0, 1, 1, 0, 1, 0, 1, 1, 1, 0, 1}
# ==========================================================================================
# Testing ODD Parity Mode
# Golden queue = '{0, 1, 1, 1, 0, 0, 0, 0, 0, 1, 1}
# Actual queue = '{0, 1, 1, 1, 0, 0, 0, 0, 0, 1, 1}
# ==========================================================================================
# Testing ODD Parity Mode
# Golden queue = '{0, 0, 0, 0, 1, 1, 0, 1, 0, 1, 1}
# Actual queue = '{0, 0, 0, 0, 1, 1, 0, 1, 0, 1, 1}
# ==========================================================================================
# Testing ODD Parity Mode
# Golden queue = '{0, 0, 0, 1, 1, 0, 0, 1, 0, 1, 1}
# Actual queue = '{0, 0, 0, 1, 1, 0, 0, 1, 0, 1, 1}
# ==========================================================================================
# Testing ODD Parity Mode
# Golden queue = '{0, 1, 1, 0, 0, 0, 0, 1, 0, 1, 1}
# Actual queue = '{0, 1, 1, 0, 0, 0, 0, 1, 0, 1, 1}
# ==========================================================================================
# Testing ODD Parity Mode
# Golden queue = '{0, 0, 0, 0, 1, 1, 0, 1, 0, 1, 1}
# Actual queue = '{0, 0, 0, 0, 1, 1, 0, 1, 0, 1, 1}
# ==========================================================================================
# Testing ODD Parity Mode
# Golden queue = '{0, 1, 1, 0, 0, 0, 0, 0, 1, 1, 1}
# Actual queue = '{0, 1, 1, 0, 0, 0, 0, 0, 1, 1, 1}
# ==========================================================================================
# Testing ODD Parity Mode
# Golden queue = '{0, 1, 1, 1, 1, 0, 1, 1, 0, 0, 1}
# Actual queue = '{0, 1, 1, 1, 1, 0, 1, 1, 0, 0, 1}
# ==========================================================================================
# Testing ODD Parity Mode
# Golden queue = '{0, 1, 1, 1, 0, 0, 1, 1, 0, 1, 1}
# Actual queue = '{0, 1, 1, 1, 0, 0, 1, 1, 0, 1, 1}
# ==========================================================================================
# Testing ODD Parity Mode
# Golden queue = '{0, 0, 0, 1, 0, 1, 0, 1, 1, 0, 1}
# Actual queue = '{0, 0, 0, 1, 0, 1, 0, 1, 1, 0, 1}
# ==========================================================================================
#
# Test for ODD Data passed
#
# ==========================================================================================
```

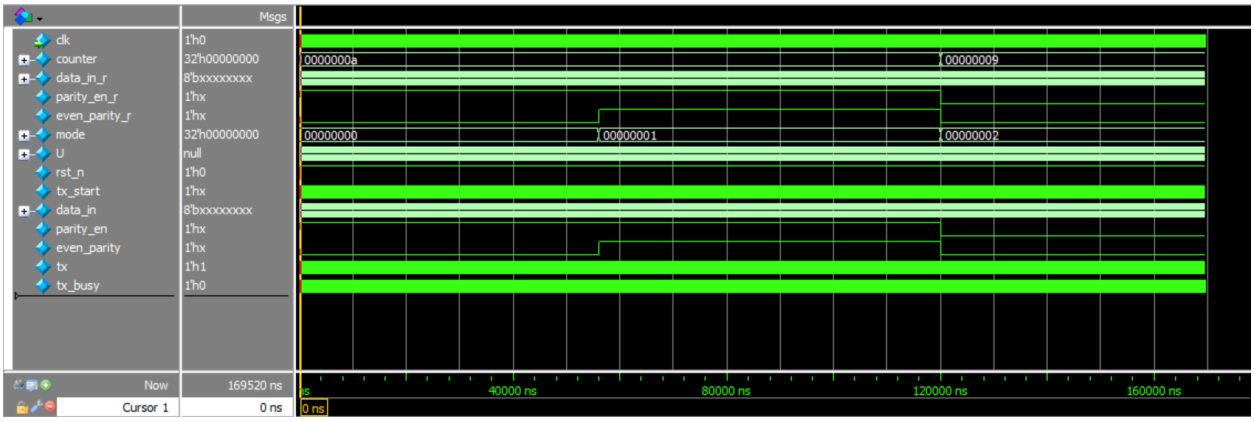
- **Snippet from EVEN PARITY Test**

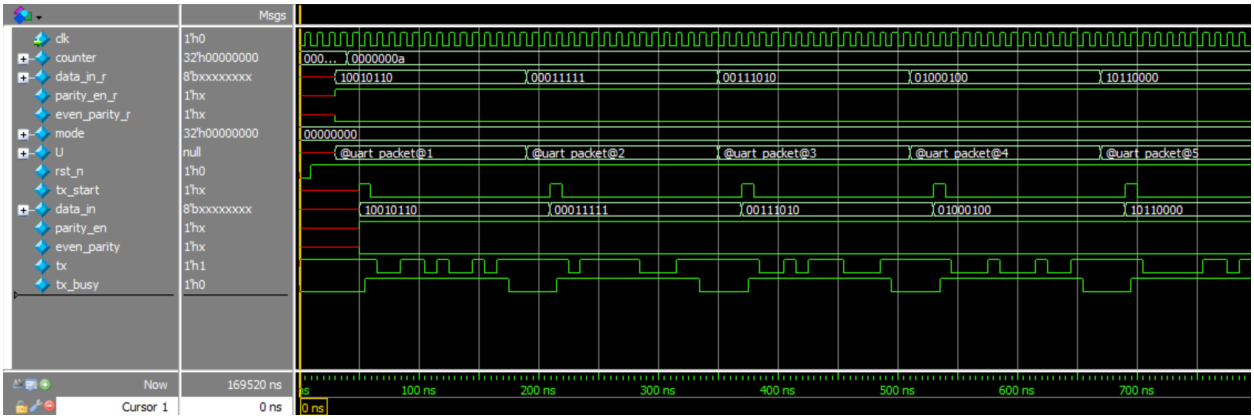
```
# Testing EVEN Parity Mode
# Golden queue = '{0, 0, 0, 1, 0, 1, 0, 0, 0, 1, 1}
# Actual queue = '{0, 0, 0, 1, 0, 1, 0, 0, 0, 1, 1}
# ==========================================================================================
# Testing EVEN Parity Mode
# Golden queue = '{0, 0, 0, 0, 1, 1, 1, 1, 0, 1, 1}
# Actual queue = '{0, 0, 0, 0, 1, 1, 1, 1, 0, 1, 1}
# ==========================================================================================
# Testing EVEN Parity Mode
# Golden queue = '{0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1}
# Actual queue = '{0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1}
# ==========================================================================================
# Testing EVEN Parity Mode
# Golden queue = '{0, 0, 1, 0, 1, 1, 0, 1, 1, 0, 1}
# Actual queue = '{0, 0, 1, 0, 1, 1, 0, 1, 1, 0, 1}
# ==========================================================================================
# Testing EVEN Parity Mode
# Golden queue = '{0, 1, 0, 1, 0, 1, 1, 0, 0, 1, 1}
# Actual queue = '{0, 1, 0, 1, 0, 1, 1, 0, 0, 1, 1}
# ==========================================================================================
# Testing EVEN Parity Mode
# Golden queue = '{0, 0, 0, 1, 0, 1, 1, 0, 1, 1, 1}
# Actual queue = '{0, 0, 0, 1, 0, 1, 1, 0, 1, 1, 1}
# ==========================================================================================
# Testing EVEN Parity Mode
# Golden queue = '{0, 0, 1, 0, 1, 1, 1, 1, 1, 1, 1}
# Actual queue = '{0, 0, 1, 0, 1, 1, 1, 1, 1, 1, 1}
# ==========================================================================================
# Testing EVEN Parity Mode
# Golden queue = '{0, 0, 0, 1, 1, 1, 0, 1, 0, 1, 1}
# Actual queue = '{0, 0, 0, 1, 1, 1, 0, 1, 0, 1, 1}
# ==========================================================================================
# Testing EVEN Parity Mode
# Golden queue = '{0, 0, 0, 1, 1, 0, 1, 1, 1, 0, 1}
# Actual queue = '{0, 0, 0, 1, 1, 0, 1, 1, 1, 0, 1}
# ==========================================================================================
# Testing EVEN Parity Mode
# Golden queue = '{0, 1, 1, 1, 0, 0, 1, 1, 0, 0, 1}
# Actual queue = '{0, 1, 1, 1, 0, 0, 1, 1, 0, 0, 1}
# ==========================================================================================
# Testing EVEN Parity Mode
# Golden queue = '{0, 1, 0, 1, 0, 1, 1, 1, 0, 0, 1}
# Actual queue = '{0, 1, 0, 1, 0, 1, 1, 1, 0, 0, 1}
# ==========================================================================================
# Testing EVEN Parity Mode
# Golden queue = '{0, 0, 1, 1, 1, 0, 0, 0, 1, 1, 1}
# Actual queue = '{0, 0, 1, 1, 1, 0, 0, 0, 1, 1, 1}
# ==========================================================================================
# Testing EVEN Parity Mode
# Golden queue = '{0, 1, 0, 0, 1, 1, 1, 1, 1, 1, 1}
# Actual queue = '{0, 1, 0, 0, 1, 1, 1, 1, 1, 1, 1}
# ==========================================================================================
# Testing EVEN Parity Mode
# Golden queue = '{0, 1, 0, 0, 1, 0, 0, 1, 0, 0, 1}
# Actual queue = '{0, 1, 0, 0, 1, 0, 0, 1, 0, 0, 1}
# ==========================================================================================
#
# Test for EVEN Data passed
#
# ==========================================================================================
```

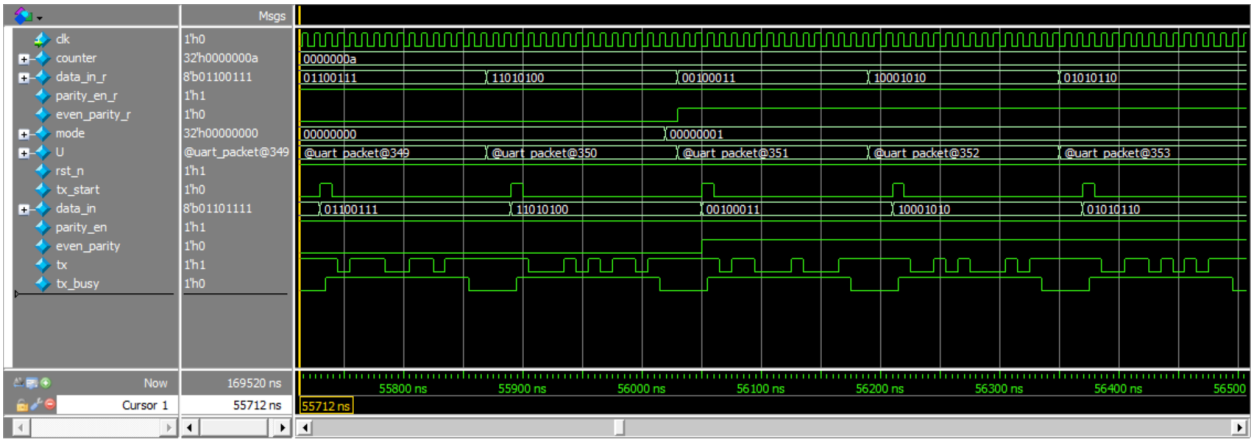
- **Snippet from NO PARITY Test**

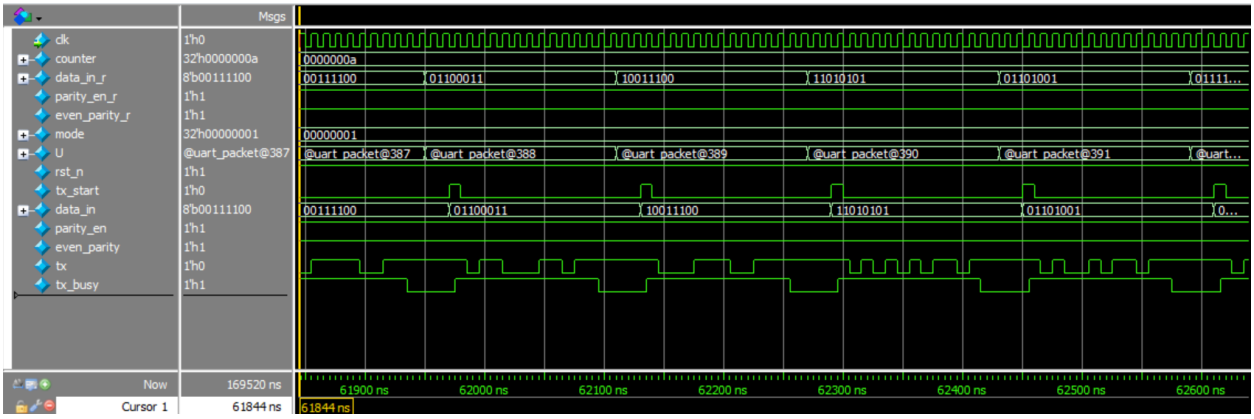
```
# Testing NO Parity Mode
# Golden queue = '{0, 1, 1, 1, 1, 0, 1, 0, 1, 1}
# Actual queue = '{0, 1, 1, 1, 1, 0, 1, 0, 1, 1}
# ==========================================================================================
# Testing NO Parity Mode
# Golden queue = '{0, 1, 0, 0, 1, 1, 0, 1, 1, 1}
# Actual queue = '{0, 1, 0, 0, 1, 1, 0, 1, 1, 1}
# ==========================================================================================
# Testing NO Parity Mode
# Golden queue = '{0, 1, 1, 1, 1, 1, 0, 0, 0, 1}
# Actual queue = '{0, 1, 1, 1, 1, 1, 0, 0, 0, 1}
# ==========================================================================================
# Testing NO Parity Mode
# Golden queue = '{0, 1, 0, 0, 0, 0, 0, 0, 0, 1}
# Actual queue = '{0, 1, 0, 0, 0, 0, 0, 0, 0, 1}
# ==========================================================================================
# Testing NO Parity Mode
# Golden queue = '{0, 0, 1, 1, 0, 0, 0, 0, 1, 1}
# Actual queue = '{0, 0, 1, 1, 0, 0, 0, 0, 1, 1}
# ==========================================================================================
# Testing NO Parity Mode
# Golden queue = '{0, 1, 1, 0, 1, 0, 1, 0, 0, 1}
# Actual queue = '{0, 1, 1, 0, 1, 0, 1, 0, 0, 1}
# ==========================================================================================
# Testing NO Parity Mode
# Golden queue = '{0, 1, 1, 0, 1, 1, 1, 0, 0, 1}
# Actual queue = '{0, 1, 1, 0, 1, 1, 1, 0, 0, 1}
# ==========================================================================================
# Testing NO Parity Mode
# Golden queue = '{0, 1, 1, 1, 1, 1, 1, 0, 1, 1}
# Actual queue = '{0, 1, 1, 1, 1, 1, 1, 0, 1, 1}
# ==========================================================================================
# Testing NO Parity Mode
# Golden queue = '{0, 0, 0, 0, 1, 1, 0, 1, 0, 1}
# Actual queue = '{0, 0, 0, 0, 1, 1, 0, 1, 0, 1}
# ==========================================================================================
# Testing NO Parity Mode
# Golden queue = '{0, 0, 1, 0, 1, 1, 1, 1, 1, 1}
# Actual queue = '{0, 0, 1, 0, 1, 1, 1, 1, 1, 1}
# ==========================================================================================
# Testing NO Parity Mode
# Golden queue = '{0, 0, 0, 0, 1, 1, 1, 0, 0, 1}
# Actual queue = '{0, 0, 0, 0, 1, 1, 1, 0, 0, 1}
# ==========================================================================================
# Testing NO Parity Mode
# Golden queue = '{0, 0, 1, 0, 1, 1, 0, 1, 0, 1}
# Actual queue = '{0, 0, 1, 0, 1, 1, 0, 1, 0, 1}
# ==========================================================================================
# Testing NO Parity Mode
# Golden queue = '{0, 1, 1, 0, 0, 1, 1, 1, 1, 1}
# Actual queue = '{0, 1, 1, 0, 0, 1, 1, 1, 1, 1}
# ==========================================================================================
#
# Test for NO PARITY Data passed
#
# ==========================================================================================
#
# The design is functioning correctly
#
# ==========================================================================================
```

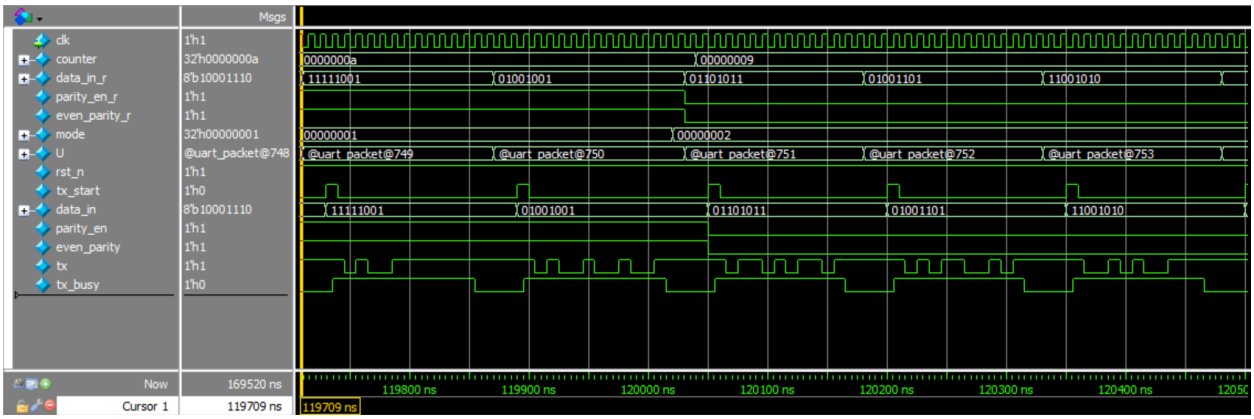
Wave Form

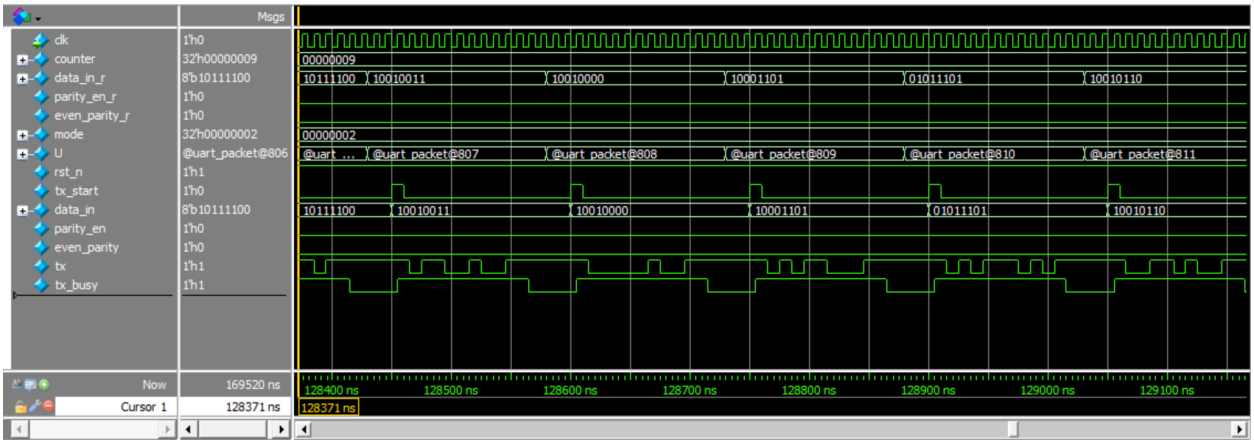

Msgs
clk 1'h0
counter 32'h00000000
data_in_r 8'bxxxxxxxx
parity_en_r 1'hx
even_parity_r 1'hx
mode 32'h00000000
U null
rst_n 1'h0
tx_start 1'hx
data_in 8'bxxxxxxxx
parity_en 1'hx
even_parity 1'hx
tx 1'h1
tx_busy 1'h0
0000000a
00000009
00000000
00000001
00000002
Now 169520 ns
Cursor 1 0 ns
40000 ns
80000 ns
120000 ns
160000 ns


Msgs
clk 1'h0
counter 32'h00000000
data_in_r 8'bxxxxxxxx
parity_en_r 1'hx
even_parity_r 1'hx
mode 32'h00000000
U null
rst_n 1'h0
tx_start 1'hx
data_in 8'bxxxxxxxx
parity_en 1'hx
even_parity 1'hx
tx 1'h1
tx_busy 1'h0
000... 0000000a
10010110
00011111
00111010
01000100
10110000
00000000
@uart_packet@1
@uart_packet@2
@uart_packet@3
@uart_packet@4
@uart_packet@5
Now 169520 ns
Cursor 1 0 ns
100 ns
200 ns
300 ns
400 ns
500 ns
600 ns
700 ns


Msgs
clk 1'h0
counter 32'h0000000a
data_in_r 8'b01100111
parity_en_r 1'h1
even_parity_r 1'h0
mode 32'h00000000
U @uart_packet@349
rst_n 1'h1
tx_start 1'h0
data_in 8'b01101111
parity_en 1'h1
even_parity 1'h0
tx 1'h1
tx_busy 1'h0
0000000a
01100111
11010100
00100011
10001010
01010110
00000000
00000001
@uart_packet@349
@uart_packet@350
@uart_packet@351
@uart_packet@352
@uart_packet@353
Now 169520 ns
Cursor 1 55712 ns
55800 ns
55900 ns
56000 ns
56100 ns
56200 ns
56300 ns
56400 ns
56500


Msgs
clk 1'h0
counter 32'h0000000a
data_in_r 8'b00111100
parity_en_r 1'h1
even_parity_r 1'h1
mode 32'h00000001
U @uart_packet@387
rst_n 1'h1
tx_start 1'h0
data_in 8'b00111100
parity_en 1'h1
even_parity 1'h1
tx 1'h0
tx_busy 1'h1
Now 169520 ns
Cursor 1 61844 ns
61900 ns
62000 ns
62100 ns
62200 ns
62300 ns
62400 ns
62500 ns
62600 ns


Msgs
clk 1'h1
counter 32'h0000000a
data_in_r 8'b10001110
parity_en_r 1'h1
even_parity_r 1'h1
mode 32'h00000001
U @uart_packet@748
rst_n 1'h1
tx_start 1'h0
data_in 8'b10001110
parity_en 1'h1
even_parity 1'h1
tx 1'h1
tx_busy 1'h0
Now 169520 ns
Cursor 1 119709 ns
119800 ns
119900 ns
120000 ns
120100 ns
120200 ns
120300 ns
120400 ns


Msgs
clk 1'h0
counter 32'h00000009
data_in_r 8'b10111100
parity_en_r 1'h0
even_parity_r 1'h0
mode 32'h00000002
U @uart_packet@806
rst_n 1'h1
tx_start 1'h0
data_in 8'b10111100
parity_en 1'h0
even_parity 1'h0
tx 1'h1
tx_busy 1'h1
Now 169520 ns
Cursor 1 128371 ns
128400 ns
128500 ns
128600 ns
128700 ns
128800 ns
128900 ns
129000 ns
129100 ns

Coverage

Functional coverage was implemented to ensure that the UART transmitter was tested under different input conditions. A covergroup was defined to monitor the randomized input data values and the selected parity mode. Separate coverpoints were used for the data input and the parity type, and cross coverage was added to track all combinations between them. This helps ensure that the design is exercised with different data patterns and parity configurations during simulation. Code coverage was also enabled, and its results are reported in the generated coverage .txt file along with the detailed functional coverage results.

Covergroups

Name	Class Type	Coverage	Goal	% of Goal	Status	Included	Merge_instances	Get_inst_coverage	Comment
/Uart_tb_sv_unit/uart_packet		100.00%							
TYPE U_cg		100.00%	100	100.00...		✓	auto(1)		
CVP U_cg::Data		100.00%	100	100.00...		✓			
CVP U_cg::Parity		100.00%	100	100.00...		✓			
CVP U_cg::data_in		100.00%	100	100.00...		✓			
CVP U_cg::enum_r		100.00%	100	100.00...		✓			
CROSS U_cg::Crossing		100.00%	100	100.00...		✓			

Do File

```
vlib work

vlog uart_tx.sv enum_pkg.sv uart_packet.sv Uart_tb.sv uart_interface.sv UART_TOP.sv +cover -covercells

vsim -voptargs=+acc work.UART_TOP -cover

add wave -position insertpoint \
sim:/UART_TOP/UART_I/clk \
sim:/UART_TOP/TB/counter \
sim:/UART_TOP/TB/data_in_r \
sim:/UART_TOP/TB/parity_en_r \
sim:/UART_TOP/TB/even_parity_r \
sim:/UART_TOP/TB/mode \
sim:/UART_TOP/TB/U \
sim:/UART_TOP/UART_I/rst_n \
sim:/UART_TOP/UART_I/tx_start \
sim:/UART_TOP/UART_I/data_in \
sim:/UART_TOP/UART_I/parity_en \
sim:/UART_TOP/UART_I/even_parity \
sim:/UART_TOP/UART_I/tx \
sim:/UART_TOP/UART_I/tx_busy

coverage save -onexit cov.ucdb

run -all

coverage report -details -output Uart_Cov.txt
```